Evolutionary Cellular Automata Based-Approach for Edge Detection

Akhil Movva

Abstract— Cellular Automatas are used for creating global behavior with local rules in many fields. Finding the rules which show the required behavior can be a difficult task, especially in real-world problems. This paper proposes an approach by using an evolutionary process to search for a specialized powerful Cellular Automata rule (CA) among the best collection of edge extraction rules in a given black-white image. This best set of local rules will asynchronously determine the future condition of CA. The Genetic Algorithm (GA) is used to search for the best CA rules that can make edge detection better realized.

Index Terms— Genetic Algorithms, Cellular Automata, Edge Detection, Image processing.

1 Introduction

Cellular Automata (CA), first proposed in the late 1940s by Ulam and Von Neumann with the goal of obtaining biological self-production models [1], [2]. Stephen Wolfram later developed the CA theory [3]. They were used in both the formal language theory and pattern recognition viewpoints.

CA consists of a normal cell grid, each of which can be in a finite number of possible states only. The state of a cell is defined by the preceding states of a surrounding cell neighborhood and is modified synchronously in discrete steps of time. The same rule applies to a finite state machine in each cell, typically defined as a rule table with an entry for every possible neighbourhood state configuration.

Cellular automata are discrete dynamical systems, and their simplicity and complex behavior have made them popular for the simulation of complex systems such as ordering, turbulence, disorder, breaking symmetry, etc. In the last 70 years, it has been subject to rigorous mathematical and physical studies. So, they have been widely used in modeling structures in fields such as physics, biology so sociology.

One of the benefits of CAs is that each image can be interpreted as a two-dimensional CA where each cell normally contains just one pixel and the pixel intensity is represented by the state of that cell [4] [5], the combination of a cell-matrix with its local interaction results in more complex evolving global behavior. That is, since each cell has an incredibly limited view of the system, at each stage localized information is propagated allowing for more global features of the overall CA system.

• *Akhil Movva is with the Concordia Universiy, Electrical and Computer Engineering. E-mail: akhilmovva6852@gmail.com.*

There are different applications in different scientific fields that involve image processing. Out of those techniques, edge detection is a fundamental tool in image segmentation and data extraction. It is used to pinpoint object boundaries within images and functions by detecting brightness discontinuities [5]. There are a variety of articles and papers published to date that typically address image processing using cellular automatons.
Furthermore, several papers addressed the processing of medical images based on a CA model.

While several algorithms have been developed for edge detection, it is still a difficult task with desirable performance to extract proper edges. We are using two-dimensional CA, which involves 3 ×3 Moore's neighborhood (nine neighborhoods) matrix that is used for adjusting states by comparing variations between a center pixel and its neighbors. After that, a Genetic Algorithm (GA) is defined to find a best-suited rule for efficiency extracting the edges in a black-white image.

This paper falls into six parts. After this introduction, Section 2 presents a basic concept of CA. Followed by Section 3, where can know how GA works. Later, we are going to see the proposed method in Section 4. Next, we can see the experimental procedure and results of how our GA evolved the behavior of CA in Section 5 and 6 respectively. Finally, in Section 7, it is going to end with the conclusion.

2 Cellular Automata

2.1 Basic Concepts

Cellular automata consist of a normal cell grid, in which each cell can have a finite number of possible states. The state of a cell at a given time step is updated in parallel and calculated by the preceding cell neighborhood states using a defined transition rule. Thus, the rules of the CA are local and consistent.

We need to define some important terms for a particular problem centered on CA, lattice geometry, neighborhood size, boundary conditions, initial conditions, state and transition rules [6]. Each of these terms are defined as one dimensional, two-dimensional and three-dimensional CA used for solving various types of problems. One-dimensional CA (1D CA) consists of linear cell arrays while two-dimensional CA (2D CA) cells are arranged in a rectangular or hexagonal grid with adjacent cell connexions, which is depicted in Figure 1 and 2 respectively.

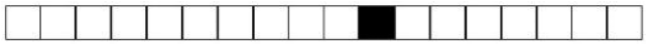


Fig-1. Structure of 1D CA.

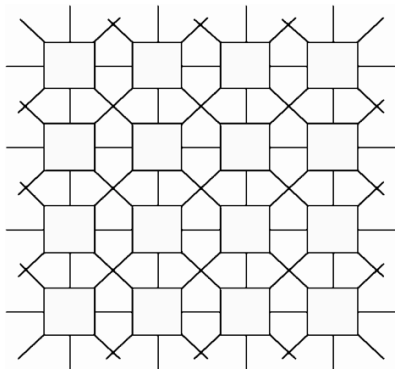
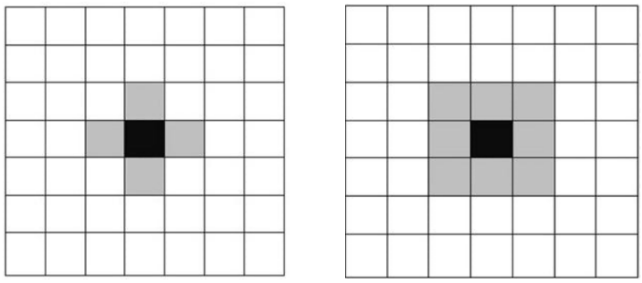


Fig-2. Structure of 2D CA.

2.2 Neighbourhood Structure

A cell's neighborhood, called the core cell (or central cell), consists of the core cell and those surrounding cells whose states decide the core cell's next condition. For cellular automates there are various neighborhood structures. The two neighborhoods most commonly used are Von Neumann and Moore neighborhood, shown in fig 3.



(a) (b)

Fig-3. Neighborhood model (a) Von Neumann, (b) Moore

We are mostly using Moore's model for our approach. Since it is having nine cells, where it consists of a center cell along with other eight neighbors.

2.3 Boundry conditions

They are many boundary conditions for Moore's neighborhood model but out of those, we going to use only periodic boundary condition. They are utilized when a transition rule is applied to the boundary cells of CA. In this condition, the extreme cells are adjacent to each other as shown in fig 4.

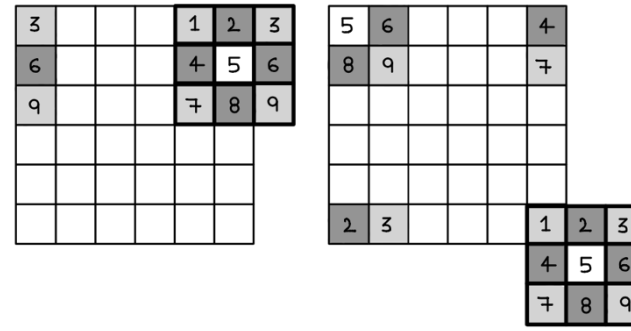


Fig-4. Boundry Cell Matrix

As mentioned in the above figure when we traverse to the extreme cells, we are going to add the beginning cells to complete the desired 3 ×3 Moore's neighborhood. When we are going left, we are wrapping around right columns and similarly when are going down, we are wrapping the top rows which is just like a circular linked list.

2.4 Rule description of 2D CA

They are only two rules {0,1} for this CA where 0 means changing the center pixel value to 0 and 1 means changing the center pixel value to 1. Each 3 ×3 Moore's neighborhood of a 2D image array is converted to a 9-bit string as shown in fig 5. And finding the appropriate rule for this string, changing the middle value according the rule and reforming the 3 ×3 array.

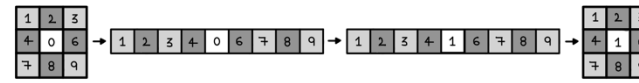


Fig-5. Converting a 3 ×3 Moore's neighborhood to 9-bit String and back to its original form

We are going to use a 9-bit string to find the rule. So, there are a total $2^9 = 512$ rules each contains a value which is either 0 or 1.

3 Genetic Algorithm

The most commonly recognized type of evolutionary algorithm is a Genetic Algorithm (GA) [7]. To find a solution using the GA, random changes to the existing solutions are made to produce new ones. They are used to find solutions in large, discrete search spaces that are too large to completely iterate. They are many ways to find

the best CA rule. But by using GA we can easily evolve to the desired rule table. In this paper we are going are compare different GA algorithms along with different instances that can get some of the best results.

How the GA works

GA operates on a population consisting of certain solutions where the number of solutions is the population size. Every solution is called an individual. Every individual solution has a chromosome. The chromosome is defined as a set of characteristics (parameters) that define the individual. Every single chromosome has a set of genes. Every gene is expressed in some way, as a string of 0s and 1s.

Each individual has a value for fitness which is used to select the best individuals. Each solution's quality is compared by using these fitnesses. The selection of the best individuals based on their quality is implemented to create what is called a mating pool where the individual of higher quality is more likely to be chosen in the mating pool.

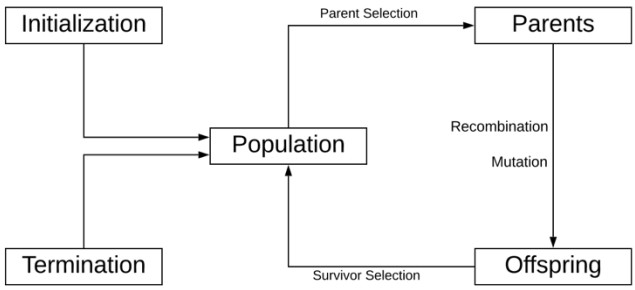


Fig-6. The general scheme of an evolutionary algorithm as a flow chart

The individuals in the mating pool are called parents and the initial selection is called parent selection. From there every two parents selected, we are going to generate two offspring (children). Each mating contains mainly two methods which are recombination (crossover) and mutation. Crossover in GA generates a new generation by naturally mutating the old generation parents in the form of carrying genes from both the parents. The number of genes carried from each parent is random (crossover rate). Whereas in mutation we are going add new gene randomly (mutation rate) which is independent of the offspring.

The set of all newly generated individuals from the mating pool will be the new population. Out of all the new and old populations, we are going to select the best parents for the next generation by using survivor selection. Each population created is called a generation. This process continues until a termination condition. Figure 6 summarizes the steps of GA as a chart.

4 Proposed Method

Problem Description

Given a randomly generated cellular automata rule table that is used to changes pixel states according to their neighboring pixels' state. In the case of a binary image, each pixel's next state is determined by looking at its neighbors in a 3 ×3 window. And also, we have a start image and a goal image. With that information, we going to write a GA that produces an image showing edges of the given image after some number of passes.

Methodology

4.1 Representation

This is the first stage in building an EA and one of the crucial stages for choosing the right representation for the problem. I am going to choose a *binary Representation* for this problem. This is the one that consists of a string of binary digits. A binary string is formed by combining the 512 rules of the individual in order. Where we are having only two rules {0,1}. On this bit string, we are going to perform our operations.

4.2 Population and Population Size

For the desired population size N, a start image, a goal image, and a set of rules table will be generated. The initial population will be stored as JSON file **CellularAutomata.json**. The individuals in the population are also JSON Strings with the following keys and values:

Start Image	Grayscale image containing values of 0 or 255
Goal Image	Grayscale image containing values of 0 or 255 (canny of start image)
Rules	9-bit truth table with output values of 0, 1

TABLE I.

Population size is used for taking diverse individuals that are sufficient for doing this task. So, I have taken the population size as *50*.

4.3 Fitness Evaluation

The fitness of an individual is calculated by *Hamming Distance*. It is nothing but comparing each pixel of goal image with the final image which we are going get after applying the truth table on the entire start image. Fitness value goes up by 1 for each difference pixel. So, the lower the fitness value the better the resulting image.

4.4 Parent Selection

Parent selection is nothing but a selection from the current generation to take part in mating [7]. I implemented three selection techniques for this problem which are Fitness-Proportionate, Exponential Rank-based, and Uniform random selection. The descriptions of these

methods are as follows:

Fitness-Proportionate Selection:

Probability of selecting an individual i for mating in a population of size μ with fitness-proportionate selection is

$$P_{FPS}(i) = f_i \Big/ \sum_{j=1}^{\mu} f_j$$

Exponential Rank-based Selection:

Probability of selecting an individual i for mating in population with exponential ranking is

$$P_{exp-rank}(i) = \frac{1 - e^{-i}}{c}$$

Where c is a normalizing constant factor according to population size.

Uniform Random Selection:

Parents are selected by uniform random distribution. So, the probability of selecting an individual i for mating in population of size μ is

$$P_{uniform}(i) = \frac{1}{\mu}$$

Uniform parent selection is unbiased- every individual has the same probability to be selected.

4.5 Variation Operations

Variation operations are used to alter an individual by using crossover and mutation. Without these operations, there won't be any significant improvement in the GA.

Crossover:

Crossover mostly combines two parents and produces two offsprings. Where each child is a combination of genes from both the parents. Here I going to use n-point and uniform crossovers, which are as follows:

- *n-Point Crossover* - Choose n random points on the two parents, split along those points and combine parts, alternating between parents to create offspring.
- *Uniform Crossover* - Each gene of the parent is changed independently based on the probability to get the first child. Whereas for the second child we need to make an inverse copy of the gene to the first child.

Mutation:

A Mutation is altering each gene independently with a probability p_m (mutation rate=0.2) for the population.

The only crossover can combine information from two parents. So, the mutation is used to introduce new information in the individual. For getting the optimum solution we need to have a proper mutation.

4.6 Survivor Selection

Managing the process of reducing the working memory of the EA from a set of μ parents and λ offspring to a set of μ individuals forming the next generation.

- *Fitness-based* replacement (Top μ-individuals) approach, we are going to choose best μ individuals out of the $(\mu + \lambda)$ population.
- *Crowding-based* replacement is an explicit approach where we are going to take two p's (parents) and the two o's (offspring) such that

  d (p1, o1) + d (p2, o2) < d (p1, o2) + d (p2, o1)

and let o1 compete with p1 and o2 compete with p2.

4.7 Diversity

Preserving diversity plays a very prominent role in the EA. So, I am going to measure the diversity by using the following function which calculates distance between all individuals:

$$D = \sum_{i=1}^{N} \sqrt{\sum_{j=1}^{i-1} d(g_i, g_j)}$$

Where $d(g_i, g_j)$ is the distance (I'm using Euclidean) between individuals g_i and g_j.

Evolutionary CA for the edge detection

In this approach, I going to find the best transition rules for the edge detection of a black-white image. The CA unit is represented by a rectangle of 9 as shown in section 2.2. Where we can also see the total 512 rules to be searched for the best CAs. Therefore, we are going to write and compare different GA algorithms that can determine the best packet of transition rules that are able to achieve the edge detection of a binary image.

```
                    ALGORITHM

Procedure for GA {
SET pop size = 50
SET gen size = 50
FOR each RUN of new Population {
  Randomly Initiate Population (pop size)
  Evaluate Population (pop size)
  While (not Gen size) {
       Select uniform parents (pop size)
       Uniform Crossover (cross rate=0.8)
       Uniform Mutation (mut rate=0.2)
       Select Top Offsprings (pop size)
       Evaluate Population (pop size)
  }
 }
}
```

5 Experimental Procedure

In this section, we are going to see the procedure for implementing the best GA algorithm (ALG-1). From section 4, we have already acknowledged different parent selection, crossover and survivor selection methods that I used for the comparison. So, here we are going to use one of each, to form the best resulting CA rule table for edge detection. Whereas remaining can also be implemented in a similar pattern.

5.1 Population Initialization

First, we are going to initialize the population based on the input image (*initial image*). As we already know, we are only implementing this algorithm on a binary image. So, we need to make sure the input image is grayscale initially and later we are thresholding the image into a binary scale which is our *start image*. We are going apply canny edge detection on this start image to find the edges and this is going to be our *goal image*.

After the initial image processing, we are going to generate our 9-bit rule table. They are $2^9 = 512$ possible binary strings with each one assigned 0 or 1. And we are going to generate these rule tables depending on the population size but keeping the start and goal images constant for the entire RUN. We are going to store these population in a JSON file for further implementation.

5.2 Evaluate Population

In this part, we are going to take the individual rule table and apply them to the start image by using the JSON file. Initially, we are using the 3 ×3 Moore's neighborhood model and convert it into a 9-bit string as mentioned in section 2. After that, we are searching for this bit-string in the rule table and replace the middle value according to the rule and convert it back into a 3 ×3 patch. This process is done on the entire start image by traversing in both horizontal and vertical directions and store these new values in a *final image*.

Now we are going to evaluate the fitness value for this final image by comparing with the goal image and assessing the *Hamming Distance* between these two images. After fitness evaluation, we are going to store the final image and corresponding fitness values for future computations.

5.3 Uniform Parent Selection

Here, we are using a uniform random selection for this algorithm. Random individuals are selected with equal probability and passed to our next process. Which is nothing but select an individual from the list and store it in a new list. But, for the next selection, we are excluding the individual already selected and continue until each and every parent gets selected.

5.4 Uniform Crossover

In the uniform crossover, we are going to select two individuals every time and each one-off its rules are stored in a string of length 512. After that, based on the crossover rate (equal to 0.8), we are going to interchange the bits for these two strings. Later we are going to pass this new population to the mutation operation.

5.5 Uniform Mutation

For this uniform mutation, we are again using the same technique as in uniform crossover but for one individual at a time based on the mutation rate (equal to 0.2). It is nothing but we are selecting one individual randomly at a time and mutating its bits before passing it through evaluate population and later to survival selection.

5.6 Survival selection

This survival selection is crucial for the next generations. So, we are first sorting and then selecting top offsprings based on fitness values. Where μ is the population size.

Finally, after reaching the termination condition or in my case generation size we are going to store the best optimal set of CA rule table.

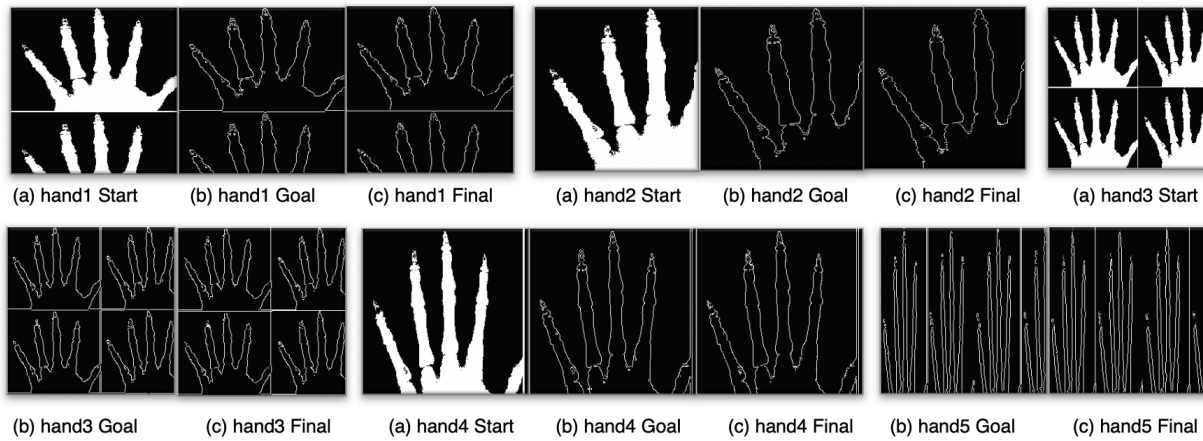


Fig-7. Edge detection of different binary hand images (256×256) (a) binary start image, (b) canny result (goal) (c) final result using powerful rule

GA	ALG-1 (Best)	ALG-2	ALG-3	ALG-4	ALG-5	ALG-6
Symbolic Parameters						
Representation	Bit-String	Bit-String	Bit-String	Bit-String	Bit-Sting	Bit-Sting
Parent Selection	Uniform Random	Exponential Rank	Fitness-Proportionate	Uniform Random	Uniform Random	Uniform Random
Recombination	Uniform	n-point	n-point	Uniform	Uniform	Uniform
Mutation	Uniform	Uniform	Uniform	Uniform	Uniform	Uniform
Survivor Selection	Top μ individuals	Top μ individuals	Crowding	Crowding	Top μ individuals	Top μ individuals
Numeric Parameters						
Population Size	50	50	50	50	70	50
Generation Size	50	50	50	50	50	70
Mutation Rate	0.2	0.2	0.2	0.2	0.1	0.5
Crossover Rate	0.8	0.8	0.8	0.8	1	0.5
Results						
Minimum Avg Fitness	878	1071	1441	1470	2516	1535

TABLE II: The above table has 4 EAs, with 3 instances for ALG-1. Out of them, ALG-1 is having slightly better performance when compared with others

6 Experimental Results

I am going to present the results (see Fig-7) of the best GA algorithm, for which I compared different algorithms along with, same algorithms different parameters as shown in TABLE II. The fitness values in the evolutionary algorithm are calculated by comparing our final image with pythons (OpenCV) default canny edge detector [8]. I tested this on python's IDE Spyder on a 2.3 GHz Quad-Core Intel Core i5 processor with 16 GB 2133 MHz LPDDR3 memory.

I have calculated the best rule table by using ALG-1 for different RUNS, where each RUN's start image is random combinations of initial image (binary and in fig-8). So, I tested my results by using the best rule table which I got from binary hand2 in fig-7 and I find out the edges of different images, just like a filter which I showed them in fig-8. Here, one more thing we notice that the horizontal lines in the mixed image are missing because the CA rule table did not train for it.

In TABLE III and TABLE IV where I compared fitness values for each hand image along with diversities. For the fitness comparison, we can see a random population's best fitness without applying GA, the first generation's best fitness and last generation best fitness. The average of the final best fitness values is 878 and from this value we can say that we almost got edge detected image from the best GA algorithm. We can also compare diversities and say that 50% of average diversity is going to be preserved for this GA algorithm.

Image Names	Best fitness before applying GA	Initial best fitness	Final best fitness
hand1	4270	2433	895
hand2	42299	1752	391
hand3	42481	4313	1408
hand4	2692	2027	664
hand5	53226	2614	1032
Average fitness		2627.8	878

TABLE III – Fitness value comparision in the binary hand images

Image Names	Initial diversity	Final diversity
hand1	74.275	37.676
hand2	74.240	40.676
hand3	74.208	35.424
hand4	74.219	35.299
hand5	74.235	33.355
Average fitness	74.235	36.486

TABLE IV – Diversity value comparision in the binary hand images

From fig-9 we can see that the mean of minimum fitness values of each RUN is continued learning (still decreasing). So, I tested this algorithm by taking another initial image (**binary Lena image** shown in fig-10); increasing population size (**pop size = 100**) and generation size (**gen size = 100**); variation operations constant. This is now called as **Modified ALG-1** (MoALG-1).

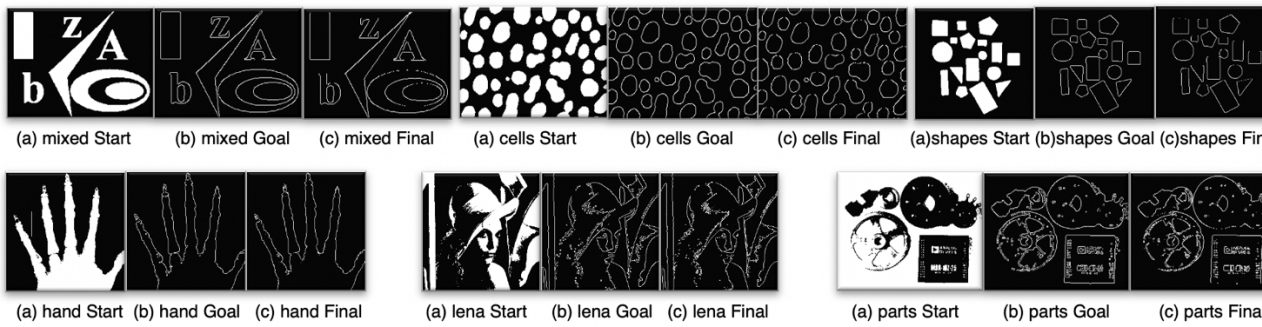
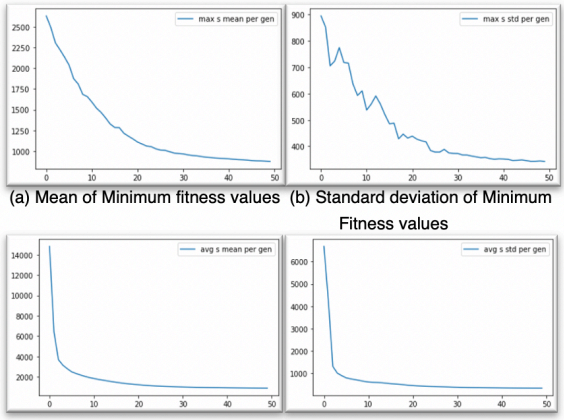


Fig-8. Edge detection of different binary images using best rule table of ALG-1 (5-RUNS) (a) binary start image, (b) canny result (goal) (c) final result using powerful rule



(a) Mean of Minimum fitness values (b) Standard deviation of Minimum Fitness values

(c) Mean of Average fitness values (d) Standard deviation of Average Fitness values

Fig-9. Mean plots for the all the 5 RUNS of binaryhand images and for each plotted figure (a), (b), (c) and (d) the x-axis is generations (50) and y-axis is fitness values.

Image Names	Results I fitness (ALG-1)	SSIM for ALG-1	Results II fitness (MoALG-1)	SSIM for MoALG-1
Shapes (256×241)	273	0.966	4	0.999
Mixed (256×192)	373	0.947	45	0.994
Binaryhand (256×256)	480	0.965	289	0.974
Cells (256×192)	661	0.931	321	0.987
Parts (256×209)	1933	0.830	1280	0.878
Lena (256×256)	2077	0.857	1377	0.893
Average fitness	966.167	0.916	552.667	0.954

TABLE V – SSIM & Fitness value comparision for the images using different best CA rule tables obtained form the binary hand and Lena.

Now, we are doing an objective evaluation by calculating the Structural SIMilarity (SSIM) index. This is a method for measuring the similarity between two images [9]. We are comparing the results (TABLE V) from the initial best rule table form the binary-hand2 image (figure 7), ALG-1 (Results-I) with the results from binary-Lena image (figure 10), MoALG-1 (Results-II) and its output edge detected images are shown in fig-10.

From the MoALG-1 results are much better and rectified our horizontal line problem in the mixed image, this is because we trained with a more complex image. We can see that the updated best CA rule table can detect edges more accurately.

In TABLE V, we see the robustness of the rule table for which every image we tested for edge detection is 95% accurate on average (This values varies on set of images). As we are getting better results in MoALG-1 but computational time is 6 times greater. The CA rule table used to get the results are shown in the **Appendix**.

7 Conclusion

In this paper, the proposed CA model applied in edge detection provides insights into how evolutionary processes can be used to detect local patterns that enable optimal edge detection of a given binary image. Indeed, one of those local structural properties was established via the GA, allowing us to study the evolutionary emergence of sophisticated computing.

As we compared different GA algorithms and used the best algorithm to find the rule table. Later, we directly applied this rule table for different images to findout and check the robustness of the edge detection. Thus, we can say that for a given global computation task, the GA evoled the best rule table to get our optimal result with an average of 95% accuracy.

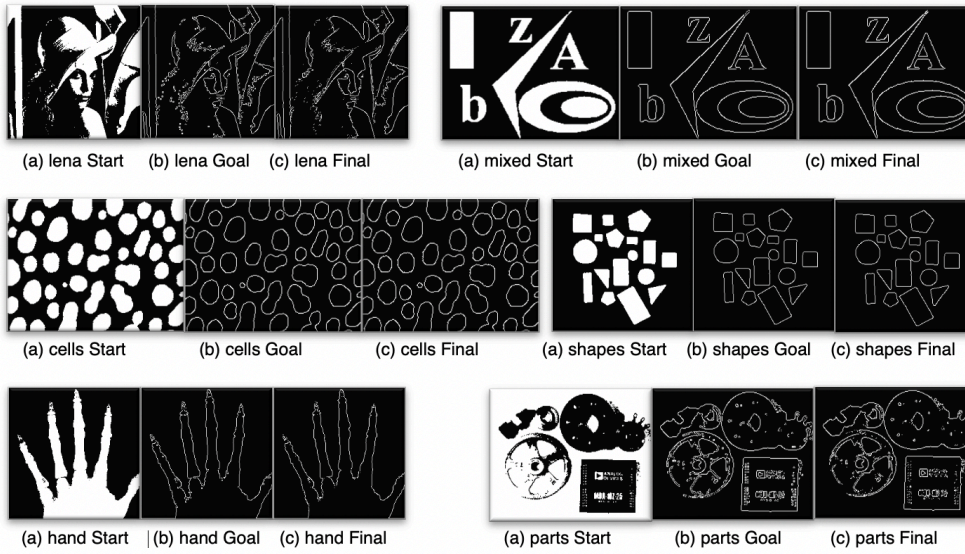


Fig-10. Edge detection of different binary images using best rule table of MoALG-1 (a) binary start image, (b) canny result (goal) (c) final result using powerful rule

Future Work

This work still needs some improvement in computational time. Whereat present for 50 individuals and 50 generations is taking roughly forty minutes to one hour. So, if we can implement parallelism in applying the rule table, we can achieve better results by testing on more parameters in less time. One other improvement we can make in GA is by using the same best CA rule table for multiple images to train. Than taking every RUN random individual, we may use one individual from the previous RUNS best CA rule table. So by using this trick, we may get better results for less amount population and generations.

Acknowledgment

I thank Nawwaf Kharma, Ph.D., PEng (Condordia University, Department of Electrical and Computer Engineering). I also thank Koa Wells, MASc (Condordia Univertsiy) and Alen Davis (Concordia University).

References

[1] Ulam, S. 1963. Some Ideas and Prospects in Biomathematics. Annual Review of Biophysics and Bioengineering. pp. 277-292.

[2] Neumann, J. V. 1966. Theory of Self-Reproducing Automata. University of Illinois Press.

[3] Wolfram, S. 1984. Computation Theory of Cellular Automata. Commun. Math. Phys. pp. 15-57.

[4] Khan, A. R., Choudhury, P. P., Dihidar, K., Mitra, S., Sarkar, P. 1997. VLSI architecture of cellular automata machine. Computers and Mathematics with Applications. 33(5). 79-94.

[5] Gonzalez, R. C. and Woods, R. E. 2002. Digital Image Processing. Second Edition. Prentice- Hall.

[6] Chang, C., Zhang, Y., Gdong, Y. 2004. Cellular Automata for Edge Detection of Images. IEEE proceedings on Machine Learning and Cybernetics. 26-29.

[7] Eiben, Agoston E., and James E. Smith. *Introduction to evolutionary computing*. Vol. 53. Berlin: springer, 2003.

[8] Canny, J.: A Computational Approach to Edge Detection. IEEE Trans. on PAMI, 8(6) (1986)

[9] Wang, Zhou, et al. "Image quality assessment: from error visibility to structural similarity." *IEEE transactions on image processing* 13.4 (2004): 600-612.

Akhil Movva recived his B.Tech degree in computer science and engineering from National Institute of Technology, Durgapur, India. He is currently pursuing his M.Eng degree in electrical and computer engineering at Concordia University, Montreal, Canada.

APPENDIX

ALG-1 - Best Rule Table, Tranined on BinaryHand2 image from figure 7

[["000000000", 0], ["000000001", 0], ["000000010", 1], ["000000011", 0], ["000000100", 0], ["000000101", 1], ["000000110", 0], ["000000111", 0], ["000001000", 1], ["000001001", 0], ["000001010", 0], ["000001011", 1], ["000001100", 0], ["000001101", 1], ["000001110", 0], ["000001111", 1], ["000010000", 1], ["000010001", 0], ["000010010", 0], ["000010011", 1], ["000010100", 0], ["000010101", 0], ["000010110", 1], ["000010111", 1], ["000011000", 0], ["000011001", 1], ["000011010", 1], ["000011011", 1], ["000011100", 1], ["000011101", 1], ["000011110", 0], ["000011111", 1], ["000100000", 1], ["000100001", 1], ["000100010", 1], ["000100011", 0], ["000100100", 0], ["000100101", 1], ["000100110", 1], ["000100111", 1], ["000101000", 1], ["000101001", 0], ["000101010", 0], ["000101011", 0], ["000101100", 1], ["000101101", 0], ["000101110", 1], ["000101111", 1], ["000110000", 1], ["000110001", 1], ["000110010", 1], ["000110011", 0], ["000110100", 1], ["000110101", 0], ["000110110", 1], ["000110111", 1], ["000111000", 1], ["000111001", 1], ["000111010", 1], ["000111011", 0], ["000111100", 1], ["000111101", 0], ["000111110", 1], ["000111111", 0], ["001000000", 0], ["001000001", 0], ["001000010", 1], ["001000011", 0], ["001000100", 0], ["001000101", 1], ["001000110", 0], ["001000111", 1], ["001001000", 0], ["001001001", 1], ["001001010", 0], ["001001011", 1], ["001001100", 1], ["001001101", 0], ["001001110", 0], ["001001111", 1], ["001010000", 0], ["001010001", 0], ["001010010", 0], ["001010011", 1], ["001010100", 0], ["001010101", 1], ["001010110", 0], ["001010111", 0], ["001011000", 1], ["001011001", 1], ["001011010", 0], ["001011011", 1], ["001011100", 1], ["001011101", 1], ["001011110", 1], ["001011111", 1], ["001100000", 0], ["001100001", 0], ["001100010", 1], ["001100011", 1], ["001100100", 0], ["001100101", 1], ["001100110", 0], ["001100111", 1], ["001101000", 1], ["001101001", 1], ["001101010", 0], ["001101011", 0], ["001101100", 0], ["001101101", 0], ["001101110", 1], ["001101111", 0], ["001110000", 1], ["001110001", 1], ["001110010", 0], ["001110011", 1], ["001110100", 0], ["001110101", 1], ["001110110", 1], ["001110111", 0], ["001111000", 0], ["001111001", 0], ["001111010", 0], ["001111011", 1], ["001111100", 0], ["001111101", 1], ["001111110", 1], ["001111111", 0], ["010000000", 0], ["010000001", 0], ["010000010", 0], ["010000011", 1], ["010000100", 0], ["010000101", 0], ["010000110", 0], ["010000111", 0], ["010001000", 0], ["010001001", 0], ["010001010", 1], ["010001011", 1], ["010001100", 1], ["010001101", 1], ["010001110", 1], ["010001111", 0], ["010010000", 0], ["010010001", 0], ["010010010", 1], ["010010011", 1], ["010010100", 1], ["010010101", 1], ["010010110", 1], ["010010111", 0], ["010011000", 0], ["010011001", 0], ["010011010", 0], ["010011011", 1], ["010011100", 0], ["010011101", 1], ["010011110", 0], ["010011111", 0], ["010100000", 1], ["010100001", 0], ["010100010", 1], ["010100011", 1], ["010100100", 1], ["010100101", 0], ["010100110", 1], ["010100111", 1], ["010101000", 0], ["010101001", 0], ["010101010", 1], ["010101011", 0], ["010101100", 1], ["010101101", 0], ["010101110", 1], ["010101111", 0], ["010110000", 1], ["010110001", 1], ["010110010", 0], ["010110011", 0], ["010110100", 1], ["010110101", 0], ["010110110", 0], ["010110111", 1], ["010111000", 0], ["010111001", 1], ["010111010", 0], ["010111011", 0], ["010111100", 0], ["010111101", 1], ["010111110", 1], ["010111111", 0], ["011000000", 0], ["011000001", 0], ["011000010", 0], ["011000011", 0], ["011000100", 1], ["011000101", 0], ["011000110", 1], ["011000111", 0], ["011001000", 1], ["011001001", 1], ["011001010", 1], ["011001011", 1], ["011001100", 0], ["011001101", 0], ["011001110", 1], ["011001111", 1], ["011010000", 1], ["011010001", 1], ["011010010", 0], ["011010011", 0], ["011010100", 0], ["011010101", 0], ["011010110", 1], ["011010111", 1], ["011011000", 1], ["011011001", 1], ["011011010", 1], ["011011011", 0], ["011011100", 1], ["011011101", 1], ["011011110", 1], ["011011111", 0], ["011100000", 0], ["011100001", 1], ["011100010", 0], ["011100011", 1], ["011100100", 1], ["011100101", 1], ["011100110", 1], ["011100111", 0], ["011101000", 1], ["011101001", 1], ["011101010", 1], ["011101011", 1], ["011101100", 0], ["011101101", 1], ["011101110", 1], ["011101111", 0], ["011110000", 1], ["011110001", 0], ["011110010", 1], ["011110011", 0], ["011110100", 0], ["011110101", 0], ["011110110", 0], ["011110111", 1], ["011111000", 1], ["011111001", 1], ["011111010", 0], ["011111011", 0], ["011111100", 0], ["011111101", 0], ["011111110", 1], ["011111111", 0], ["100000000", 0], ["100000001", 1], ["100000010", 0], ["100000011", 0], ["100000100", 0], ["100000101", 0], ["100000110", 0], ["100000111", 0], ["100001000", 0], ["100001001", 0], ["100001010", 0], ["100001011", 1], ["100001100", 1], ["100001101", 0], ["100001110", 1], ["100001111", 0], ["100010000", 0], ["100010001", 1], ["100010010", 0], ["100010011", 1], ["100010100", 1], ["100010101", 0], ["100010110", 0], ["100010111", 0], ["100011000", 1], ["100011001", 1], ["100011010", 1], ["100011011", 0], ["100011100", 0], ["100011101", 1], ["100011110", 0], ["100011111", 0], ["100100000", 0], ["100100001", 1], ["100100010", 1], ["100100011", 1], ["100100100", 0], ["100100101", 0], ["100100110", 1], ["100100111", 1], ["100101000", 0], ["100101001", 0], ["100101010", 0], ["100101011", 0], ["100101100", 0], ["100101101", 0], ["100101110", 0], ["100101111", 0], ["100110000", 0], ["100110001", 1], ["100110010", 0], ["100110011", 1], ["100110100", 0], ["100110101", 1], ["100110110", 1], ["100110111", 1], ["100111000", 0], ["100111001", 0], ["100111010", 0], ["100111011", 0], ["100111100", 0], ["100111101", 1], ["100111110", 0], ["100111111", 0], ["101000000", 1], ["101000001", 1], ["101000010", 1], ["101000011", 0], ["101000100", 1], ["101000101", 0], ["101000110", 1], ["101000111", 0], ["101001000", 0], ["101001001", 0], ["101001010", 1], ["101001011", 1], ["101001100", 1], ["101001101", 1], ["101001110", 0], ["101001111", 1], ["101010000", 1], ["101010001", 1], ["101010010", 0], ["101010011", 0], ["101010100", 0], ["101010101", 1], ["101010110", 1], ["101010111", 0], ["101011000", 0], ["101011001", 1], ["101011010", 1], ["101011011", 1], ["101011100", 0], ["101011101", 1], ["101011110", 0], ["101011111", 1], ["101100000", 1], ["101100001", 0], ["101100010", 1], ["101100011", 0], ["101100100", 0], ["101100101", 0], ["101100110", 0], ["101100111", 0], ["101101000", 0], ["101101001", 0], ["101101010", 0], ["101101011", 1], ["101101100", 0], ["101101101", 1], ["101101110", 0], ["101101111", 0], ["101110000", 1], ["101110001", 0], ["101110010", 1], ["101110011", 1], ["101110100", 1], ["101110101", 0], ["101110110", 1], ["101110111", 0], ["101111000", 0], ["101111001", 1], ["101111010", 1], ["101111011", 1], ["101111100", 1], ["101111101", 0], ["101111110", 0], ["101111111", 0], ["110000000", 0], ["110000001", 0], ["110000010", 0], ["110000011", 1], ["110000100", 1], ["110000101", 0], ["110000110", 0], ["110000111", 1], ["110001000", 0], ["110001001", 0], ["110001010", 1], ["110001011", 1], ["110001100", 1], ["110001101", 0], ["110001110", 1], ["110001111", 0], ["110010000", 0], ["110010001", 1], ["110010010", 1], ["110010011", 0], ["110010100", 1], ["110010101", 1], ["110010110", 0], ["110010111", 1], ["110011000", 0], ["110011001", 1], ["110011010", 1], ["110011011", 0], ["110011100", 1], ["110011101", 0], ["110011110", 1], ["110011111", 0], ["110100000", 0], ["110100001", 0], ["110100010", 0], ["110100011", 0], ["110100100", 1], ["110100101", 0], ["110100110", 0], ["110100111", 1], ["110101000", 1], ["110101001", 0], ["110101010", 0], ["110101011", 1], ["110101100", 0], ["110101101", 0], ["110101110", 1], ["110101111", 0], ["110110000", 1], ["110110001", 1], ["110110010", 0], ["110110011", 1], ["110110100", 1], ["110110101", 1], ["110110110", 1], ["110110111", 0], ["110111000", 0], ["110111001", 0], ["110111010", 0], ["110111011", 0], ["110111100", 1], ["110111101", 0], ["110111110", 0], ["110111111", 0], ["111000000", 0], ["111000001", 0], ["111000010", 0], ["111000011", 1], ["111000100", 1], ["111000101", 1], ["111000110", 0], ["111000111", 0], ["111001000", 1], ["111001001", 1], ["111001010", 0], ["111001011", 0], ["111001100", 1], ["111001101", 0], ["111001110", 0], ["111001111", 0], ["111010000", 0], ["111010001", 0], ["111010010", 0], ["111010011", 0], ["111010100", 1], ["111010101", 0], ["111010110", 1], ["111010111", 1], ["111011000", 1], ["111011001", 1], ["111011010", 0], ["111011011", 0], ["111011100", 1], ["111011101", 1], ["111011110", 1], ["111011111", 0], ["111100000", 1], ["111100001", 0], ["111100010", 1], ["111100011", 0], ["111100100", 1], ["111100101", 0], ["111100110", 0], ["111100111", 0], ["111101000", 0], ["111101001", 1], ["111101010", 1], ["111101011", 0], ["111101100", 1], ["111101101", 0], ["111101110", 1], ["111101111", 0], ["111110000", 1], ["111110001", 1], ["111110010", 1], ["111110011", 0], ["111110100", 1], ["111110101", 1], ["111110110", 0], ["111110111", 1], ["111111000", 1], ["111111001", 0], ["111111010", 0], ["111111011", 0], ["111111100", 0], ["111111101", 0], ["111111110", 0], ["111111111", 0]]

Modified ALG-1 - Best Rule Table, Tranined on Lena image from figure 10

[["000000000", 0], ["000000001", 0], ["000000010", 1], ["000000011", 0], ["000000100", 0], ["000000101", 0], ["000000110", 0], ["000000111", 1], ["000001000", 1], ["000001001", 0],
["000001010", 0], ["000001011", 1], ["000001100", 0], ["000001101", 0], ["000001110", 1], ["000001111", 1], ["000010000", 0], ["000010001", 0], ["000010010", 0], ["000010011", 1],
["000010100", 0], ["000010101", 1], ["000010110", 1], ["000010111", 1], ["000011000", 0], ["000011001", 1], ["000011010", 0], ["000011011", 1], ["000011100", 1], ["000011101", 0],
["000011110", 0], ["000011111", 1], ["000100000", 0], ["000100001", 0], ["000100010", 0], ["000100011", 1], ["000100100", 0], ["000100101", 0], ["000100110", 1], ["000100111", 1],
["000101000", 0], ["000101001", 0], ["000101010", 1], ["000101011", 1], ["000101100", 0], ["000101101", 0], ["000101110", 0], ["000101111", 1], ["000110000", 0], ["000110001", 0],
["000110010", 0], ["000110011", 0], ["000110100", 0], ["000110101", 0], ["000110110", 1], ["000110111", 1], ["000111000", 0], ["000111001", 1], ["000111010", 0], ["000111011", 0],
["000111100", 1], ["000111101", 1], ["000111110", 0], ["000111111", 0], ["001000000", 0], ["001000001", 0], ["001000010", 0], ["001000011", 0], ["001000100", 0], ["001000101", 0],
["001000110", 0], ["001000111", 1], ["001001000", 0], ["001001001", 1], ["001001010", 1], ["001001011", 1], ["001001100", 0], ["001001101", 0], ["001001110", 1], ["001001111", 1],
["001010000", 0], ["001010001", 0], ["001010010", 0], ["001010011", 0], ["001010100", 0], ["001010101", 0], ["001010110", 0], ["001010111", 1], ["001011000", 1], ["001011001", 1],
["001011010", 0], ["001011011", 1], ["001011100", 0], ["001011101", 0], ["001011110", 0], ["001011111", 1], ["001100000", 0], ["001100001", 0], ["001100010", 1], ["001100011", 0],
["001100100", 0], ["001100101", 1], ["001100110", 1], ["001100111", 0], ["001101000", 0], ["001101001", 0], ["001101010", 1], ["001101011", 1], ["001101100", 0], ["001101101", 0],
["001101110", 0], ["001101111", 0], ["001110000", 0], ["001110001", 1], ["001110010", 1], ["001110011", 1], ["001110100", 0], ["001110101", 1], ["001110110", 0], ["001110111", 1],
["001111000", 0], ["001111001", 0], ["001111010", 1], ["001111011", 0], ["001111100", 0], ["001111101", 1], ["001111110", 0], ["001111111", 0], ["010000000", 0], ["010000001", 0],
["010000010", 0], ["010000011", 0], ["010000100", 0], ["010000101", 0], ["010000110", 1], ["010000111", 0], ["010001000", 0], ["010001001", 1], ["010001010", 0], ["010001011", 1],
["010001100", 0], ["010001101", 0], ["010001110", 0], ["010001111", 1], ["010010000", 0], ["010010001", 0], ["010010010", 0], ["010010011", 0], ["010010100", 0], ["010010101", 0],
["010010110", 0], ["010010111", 0], ["010011000", 1], ["010011001", 0], ["010011010", 1], ["010011011", 0], ["010011100", 1], ["010011101", 0], ["010011110", 1], ["010011111", 0],
["010100000", 0], ["010100001", 0], ["010100010", 1], ["010100011", 1], ["010100100", 1], ["010100101", 0], ["010100110", 0], ["010100111", 0], ["010101000", 0], ["010101001", 0],
["010101010", 0], ["010101011", 0], ["010101100", 1], ["010101101", 0], ["010101110", 1], ["010101111", 0], ["010110000", 0], ["010110001", 1], ["010110010", 0], ["010110011", 0],
["010110100", 0], ["010110101", 0], ["010110110", 0], ["010110111", 1], ["010111000", 1], ["010111001", 0], ["010111010", 1], ["010111011", 1], ["010111100", 0], ["010111101", 0],
["010111110", 1], ["010111111", 0], ["011000000", 0], ["011000001", 1], ["011000010", 1], ["011000011", 0], ["011000100", 0], ["011000101", 0], ["011000110", 0], ["011000111", 0],
["011001000", 1], ["011001001", 1], ["011001010", 1], ["011001011", 1], ["011001100", 0], ["011001101", 1], ["011001110", 0], ["011001111", 1], ["011010000", 0], ["011010001", 1],
["011010010", 0], ["011010011", 0], ["011010100", 0], ["011010101", 1], ["011010110", 0], ["011010111", 0], ["011011000", 1], ["011011001", 1], ["011011010", 0], ["011011011", 0],
["011011100", 1], ["011011101", 0], ["011011110", 1], ["011011111", 0], ["011100000", 1], ["011100001", 0], ["011100010", 0], ["011100011", 0], ["011100100", 1], ["011100101", 0],
["011100110", 1], ["011100111", 0], ["011101000", 0], ["011101001", 0], ["011101010", 1], ["011101011", 0], ["011101100", 0], ["011101101", 0], ["011101110", 0], ["011101111", 0],
["011110000", 0], ["011110001", 0], ["011110010", 1], ["011110011", 0], ["011110100", 0], ["011110101", 1], ["011110110", 0], ["011110111", 0], ["011111000", 1], ["011111001", 1],
["011111010", 1], ["011111011", 0], ["011111100", 0], ["011111101", 0], ["011111110", 0], ["011111111", 0], ["100000000", 0], ["100000001", 0], ["100000010", 0], ["100000011", 0],
["100000100", 0], ["100000101", 0], ["100000110", 0], ["100000111", 1], ["100001000", 0], ["100001001", 0], ["100001010", 0], ["100001011", 0], ["100001100", 1], ["100001101", 1],
["100001110", 1], ["100001111", 1], ["100010000", 0], ["100010001", 0], ["100010010", 0], ["100010011", 0], ["100010100", 0], ["100010101", 1], ["100010110", 1], ["100010111", 1],
["100011000", 0], ["100011001", 0], ["100011010", 0], ["100011011", 0], ["100011100", 1], ["100011101", 0], ["100011110", 0], ["100011111", 1], ["100100000", 0], ["100100001", 0],
["100100010", 0], ["100100011", 0], ["100100100", 0], ["100100101", 0], ["100100110", 1], ["100100111", 1], ["100101000", 0], ["100101001", 0], ["100101010", 1], ["100101011", 1],
["100101100", 1], ["100101101", 0], ["100101110", 1], ["100101111", 1], ["100110000", 0], ["100110001", 0], ["100110010", 0], ["100110011", 0], ["100110100", 1], ["100110101", 0],
["100110110", 1], ["100110111", 1], ["100111000", 0], ["100111001", 0], ["100111010", 0], ["100111011", 0], ["100111100", 0], ["100111101", 0], ["100111110", 0], ["100111111", 0],
["101000000", 0], ["101000001", 0], ["101000010", 1], ["101000011", 1], ["101000100", 0], ["101000101", 1], ["101000110", 0], ["101000111", 1], ["101001000", 0], ["101001001", 0],
["101001010", 1], ["101001011", 1], ["101001100", 0], ["101001101", 0], ["101001110", 0], ["101001111", 1], ["101010000", 1], ["101010001", 1], ["101010010", 0], ["101010011", 1],
["101010100", 0], ["101010101", 0], ["101010110", 0], ["101010111", 1], ["101011000", 0], ["101011001", 1], ["101011010", 1], ["101011011", 1], ["101011100", 1], ["101011101", 0],
["101011110", 0], ["101011111", 0], ["101100000", 1], ["101100001", 0], ["101100010", 1], ["101100011", 0], ["101100100", 1], ["101100101", 0], ["101100110", 0], ["101100111", 0],
["101101000", 0], ["101101001", 0], ["101101010", 0], ["101101011", 0], ["101101100", 0], ["101101101", 0], ["101101110", 0], ["101101111", 0], ["101110000", 0], ["101110001", 0],
["101110010", 0], ["101110011", 1], ["101110100", 1], ["101110101", 1], ["101110110", 1], ["101110111", 0], ["101111000", 0], ["101111001", 1], ["101111010", 1], ["101111011", 0],
["101111100", 0], ["101111101", 1], ["101111110", 0], ["101111111", 0], ["110000000", 0], ["110000001", 0], ["110000010", 1], ["110000011", 0], ["110000100", 0], ["110000101", 1],
["110000110", 0], ["110000111", 1], ["110001000", 1], ["110001001", 1], ["110001010", 0], ["110001011", 0], ["110001100", 0], ["110001101", 1], ["110001110", 1], ["110001111", 0],
["110010000", 0], ["110010001", 0], ["110010010", 0], ["110010011", 0], ["110010100", 1], ["110010101", 1], ["110010110", 1], ["110010111", 0], ["110011000", 1], ["110011001", 0],
["110011010", 1], ["110011011", 0], ["110011100", 1], ["110011101", 0], ["110011110", 1], ["110011111", 0], ["110100000", 1], ["110100001", 0], ["110100010", 0], ["110100011", 0],
["110100100", 1], ["110100101", 0], ["110100110", 1], ["110100111", 1], ["110101000", 0], ["110101001", 1], ["110101010", 0], ["110101011", 1], ["110101100", 0], ["110101101", 0],
["110101110", 0], ["110101111", 0], ["110110000", 1], ["110110001", 0], ["110110010", 0], ["110110011", 0], ["110110100", 1], ["110110101", 1], ["110110110", 1], ["110110111", 0],
["110111000", 0], ["110111001", 0], ["110111010", 0], ["110111011", 0], ["110111100", 1], ["110111101", 1], ["110111110", 0], ["110111111", 0], ["111000000", 0], ["111000001", 0],
["111000010", 0], ["111000011", 1], ["111000100", 0], ["111000101", 0], ["111000110", 0], ["111000111", 0], ["111001000", 1], ["111001001", 1], ["111001010", 1], ["111001011", 0],
["111001100", 0], ["111001101", 0], ["111001110", 0], ["111001111", 1], ["111010000", 1], ["111010001", 1], ["111010010", 1], ["111010011", 0], ["111010100", 1], ["111010101", 0],
["111010110", 1], ["111010111", 0], ["111011000", 1], ["111011001", 1], ["111011010", 0], ["111011011", 0], ["111011100", 1], ["111011101", 0], ["111011110", 1], ["111011111", 0],
["111100000", 1], ["111100001", 0], ["111100010", 1], ["111100011", 0], ["111100100", 1], ["111100101", 0], ["111100110", 1], ["111100111", 0], ["111101000", 1], ["111101001", 1],
["111101010", 0], ["111101011", 0], ["111101100", 1], ["111101101", 0], ["111101110", 0], ["111101111", 0], ["111110000", 1], ["111110001", 1], ["111110010", 0], ["111110011", 0],
["111110100", 1], ["111110101", 0], ["111110110", 0], ["111110111", 0], ["111111000", 1], ["111111001", 0], ["111111010", 0], ["111111011", 0], ["111111100", 0], ["111111101", 1],
["111111110", 0], ["111111111", 0]]